The Star

VOL. 1, NO. 004

2026-03-30

FEATURES

Exploring Regular Expressions, Part II: Regular Languages and Finite-State Automata

Reginald Braithwaite · Reginald Braithwaite (raganwald)

This is Part II of "Exploring Regular Expressions." If you haven't already, you may want to read Part I first, where we wrote a compiler that translates formal regular expressions into finite-state recognizers .

You may also want another look at the essay, A Brutal Look at Balanced Parentheses, Computing Machines, and Pushdown Automata . It covers the concepts behind finite-state machines and the the kinds of "languages" they can and cannot recognize.

Table of Contents

The Essentials from Part I

If you're familiar with formal regular expressions, and are very comfortable with the code we presented in Part I , or just plain impatient, you can skip ahead to Beyond Formal Regular Expressions .

But for those who want a refresher, we'll quickly recap regular expressions and the code we have so far.

In Part I , and again in this essay, we will spend a lot of time talking about formal regular expressions . Formal regular expressions are a minimal way to describe "regular" languages, and serve as the building blocks for the regexen we find in most programming languages.

Formal regular expressions describe languages as sets of sentences. The three basic building blocks for formal regular expressions are the empty set, the empty string, and literal symbols:

The symbol ∅ describes the language with no sentences, { } , also called "the empty set."

The symbol ε describes the language containing only the empty string, { '' } .

Literals such as x , y , or z describe languages containing single sentences, containing single symbols. e.g. The literal r describes the language { 'r' } .

What makes formal regular expressions powerful, is that we have operators for alternating, catenating, and quantifying regular expressions. Given that x is a regular expression describing some language X , and y is a regular expression describing some language Y :

The expression x | y describes the union of the languages X and Y , meaning, the sentence w belongs to x|y if and only if w belongs to the language X , or w belongs to the language Y . We can also say that x | y represents the alternation of x and y .

The expression xy describes the language XY , where a sentence ab belongs to the language XY if and only if a belongs to the language X , and b belongs to the language Y . We can also say that xy represents the catenation of the expressions x and y .

The expression x * describes the language Z , where the sentence ε (the empty string) belongs to Z , and, the sentence pq belongs to Z if and only if p is a sentence belonging to X , and q is a sentence belonging to Z . We can also say that x * represents a quantification of x .

Before we add the last rule for regular expressions, let's clarify these three rules with some examples. Given the constants a , b , and c , resolving to the languages { 'a' } , { 'b' } , and { 'b' } :

Anime vs. Marvel/DC: Designing Digital Products With Emotion In Flow

Alan Cohen · Smashing Magazine

Design isn't only pixels and patterns. It's pacing and feelings, too. Some products feel cinematic as they guide us through uncertainty, relief, confidence, and calm without yanking us around. That's Emotion in Flow . Others undercut their own moments with a joke in the wrong place, a surprise pop-up, or a jumpy transition. That's Emotion in Conflict .

These aren't UX-only ideas. You can see them everywhere in entertainment. And the clearest way to feel the difference is to compare how anime handles emotional shifts versus how Marvel and DC films stumble. We'll use two specific examples, one from Dan da Dan (anime series on Netflix) and one from James Gunn's Superman movie, to define the two concepts, and then translate them into practical product design patterns you can apply right away.

Note : We'll focus on digital products , including apps, SaaS, and web.

Emotion In Flow (Anime: Dan da Dan)

In Dan da Dan , the tonal range is wild, horror, comedy, tenderness, yet it flows .

Example: In one arc, the protagonists are on a bizarre, comedic quest involving the "golden genitals" of one of the main characters (yes, really), and in another, we're drawn into a heartbreaking story of a mother whose child is kidnapped. On paper, that shift should be a car crash. On screen, it's coherent and emotionally legible.

Why does this work on screen?

Continuity of stakes. Even when a gag lands, the characters' goals and danger stay intact. Humor releases tension after a mini-resolution; it doesn't deny the threat.

Clear mood cues. Music, framing, pacing, and character reactions telegraph the next feeling. You're primed for the shift, so you ride it rather than getting yanked.

One emotional anchor. Relationships remain the North Star, so the scene's heart doesn't get lost when the tone moves.

How does this translate to UX?

Good products do the same: prepare , transition , resolve , so users stay immersed as the emotional tone shifts.

Emotion In Conflict (Marvel/DC: James Gunn's Superman)

Lois & Clark are having a heartfelt, intimate conversation, a slow, human moment, while in the background a running gag plays out (a monster getting clobbered with a giant baseball bat). The gag steals the focus right when the scene asks you to feel something real. The result is a tonal clash that punctures the emotion instead of releasing it.

Why does this fail on screen?

Increased cognitive load. What's happening here maps directly to cognitive load theory. When a scene (or interface) asks users to process two competing emotional signals at once, it introduces extraneous cognitive load , mental effort that has nothing to do with the task or moment itself. Instead of focusing on the emotional beat, attention is split between signals that don't resolve each other. In products, this is what happens when humor, promotions, or unexpected UI changes intrude on high-stakes moments: users are forced to interpret tone and intent at the same time they're trying to act, which slows comprehension and increases stress.

Competing beats at the same time. The joke overlaps the climax of a serious beat; the audience pays attention to the switch rather than the feeling.

No tonal handoff. There's no transition that lands the intimacy before humor arrives, so the moment feels undercut rather than resolved.

How does this translate to UX?

In products, this is the confetti-before-confirmation problem, the cheeky error in a money flow, or the promo modal that appears right in the middle of a critical task. This also spikes cognitive load: users must process the humor while trying to fix a problem, which slows them down and increases stress.

Emotion in Flow Emotional shifts feel earned, telegraphed, and timed so they resolve prior beats. Immersion holds.

Emotion in Conflict A jarring switch (or hard cut) that punctures a live emotional beat. Immersion breaks.

Now that we've named it: how does this connect to UX?

How Emotions Shape Product Memorability

People don't remember the average of an experience; they remember peaks and the ending. If your flow's peak is frustration, or your ending is messy, that's what sticks. So design the emotional curve on purpose.

Emotions live across three layers (from Don Norman's Emotional Design), and your product needs to line them up:

Visceral (gut): First-impression signals: visuals, motion, haptics, sound. Examples: A steady skeleton loader calms more than a jittery spinner; a gentle success chime/haptic tap lets the win land without shouting; consistent easing/ direction tells the eye what changed.

Behavioral (doing): Can I complete my task smoothly? Fric-

From Higher-Order Functions to Libraries And Frameworks

Reginald Braithwaite · Reginald Braithwaite (raganwald)

In this essay, we will take a look at some higher-order functions, with an eye to seeing how they can be used to make our programs more expressive, while balancing that against the need to limit the perceived complexity of our programs.

introduction: expressiveness

One of the most basic ideas in programming is that functions can invoke other functions. 1

When a function invokes other functions, and when one function can be invoked by more than one other function, we have a very good thing. When we have a many-to-many relationship between functions, we have a more expressive power than when we have a one-to-many relationship. We have the ability to give each function a single responsibility, and name that responsibility. We also have the ability to ensure that one and only one function has that responsibility.

A many-to-many relationship between functions is what enables us to create a one-to-one relationship between functions and responsibilities.

Programmers often speak of languages as being expressive . Although there is no single universal definition for this word, most programmers agree that an important aspect of "expressiveness" is that the language makes it easy to write programs that are not unnecessarily verbose. If functions have many responsibilities, they become large and unwieldy. If the same responsibility needs to be implemented more than once, there is de facto redundancy.

Programs where functions have single responsibilities, and where responsibilities are implemented by single functions, avoid unnecessary verbosity.

Thus, facilitating the many-to-many relationship between functions makes it possible to write programs that are more expressive than those that do not have a many-to-many relationship between functions.

the dark side: perceived complexity

However, "With great power comes great responsibility." 2 The downside of a many-to-many relationship between functions is that the 'space of things a program might do' grows very rapidly as the size increases. "Expressiveness" is often in tension with "Perceived Complexity."

One way to think about this by analogy is to imagine we are drawing a graph. Each function is a vertex, and the calling relationship between them is an edge. Assuming that there is no "dead code," every structured program forms a connected graph .

Given a known number of nodes, the number of different ways to draw a connected graph between them is the A001187 integer sequence. Its first eleven terms are: 1, 1, 1, 4, 38, 728, 26704, 1866256, 251548592, 66296291072, 34496488594816 . Meaning that there are more than thirty-four trillion ways to organize a program with just ten functions.

This explosion of flexibility is so great that programmers have to temper it. The benefits of creating one-to-one relationships between functions and responsibilities can become overwhelmed by the difficulty of understanding programs with unconstrained potential complexity.

JavaScript has tools to help. Its blocks create namespaces, and so do its formal modules. It may soon have private object properties.

Namespaces constrain large graphs into many smaller graphs, each of which has a constrained set of ways they can be connected to other graphs. It's still a large graph, but the number of possible ways to draw it is smaller, and by analogy, it is easier to sort out what it does, and how.

What we have described is a heuristic for designing good software systems: Provide the flexibility to use many-to-many relationships between entities, while simultaneously providing ways for programmers to intentionally limit the ways that entities can be connected .

But notice that we're not saying that one mechanism does both jobs. No, we're saying that one tool helps us increase expressivity, while another helps us limit the perceived complexity of our programs, and the two work in tension with each other.

Now that we've established our heuristic, let's look at some higher-order functions, and see what they can tell us about expressiveness and perceived complexity.

higher-order functions

Functions that accept functions as parameters, and/or return functions as values, are called Higher-Order Functions , or "HOFs." Languages that support HOFs also support the idea of functions as first-class values , and nearly always support the idea of dynamically creating functions .

HOFs give programmers even more ways to decompose and compose programs, and thus more ways to write programs where there is a one-to-one relationship between functions and responsibilities. Let's look at an example.

Rumour has it that there are excellent companies that ask coöp students to write code as part of the interview process. A typical problem will ask the student to demonstrate their facility solving a problem that ought to be familiar to a computer science or computer engineering student.

For example, merging two sorted lists. This is something a student will have at least looked at, and it does have some applicability to modern service architectures. Here's a naïve implementation:

Ghost CMS & Linux - Fixing “No Space Left on Device” Issue

Dean Hume

A few years ago, I transitioned my blog from a custom ASP. NET website to Ghost CMS . I’ve been really happy with Ghost - it’s easy to set up and get running, and the Ghost community is really great.

Life often gets in the way of blogging, and I haven’t made any new posts to this blog for a while. I’ve had a few on and off issues with Amazon EC2 Linux instances and this blog over time, but generally things were working as expected. The blog has largely remained untouched for a year or so. Which is why I was quite surprised to found out that I was greeted with an annoying HTTP 503 error on my site over the Christmas period.

I thought this might just be an issue with the site, so I tried the usual Stop Instance and Restart Instance . That didn’t help.

After taking a closer look at the logs on Amazon, it turns out that I had actually run out of disk space on the instance. This is a bit weird considering I had 10 GB assigned to the volume - after all, this is only a small blog!

Increasing the size of the volume

My first thoughts were to get the site back up and running by increasing the size of the volume (or disk space) assigned to the instance. You can do this from the EC2 Management Console by selecting the instance, choosing Storage and the clicking on the Volume ID (highlighted in yellow below).

Select the Volume to update

Next, select from the Actions drop down menu and choose Modify Volume .

Choose the new size of the volume and select OK.

Once you’ve increased the size of the volume, it turns out that there is still one more step that needs to take place. You need to tell the partition to use the “new space” that you’ve just given it. Without doing this, it is still assigned the old volume size and you haven’t actually made use of the new space you’ve given it.

You can see this by SSH’ing into the instance and typing lsblk in the terminal.

NAME MAJ: MIN RM SIZE RO TYPE MOUNTPOINT xvda 202:0 0 40G 0 disk └─xvda1 202:1 0 10G 0 part / loop1 7:1 0 97.9M 1 loop /snap/core/10444 loop3 7:3 0 97.9M 1 loop /snap/core/10577 loop4 7:4 0 55.4M 1 loop /snap/core18/1932 loop6 7:6 0 55.4M 1 loop /snap/core18/1944

In my case the partition xvda1 is still assigned 10 GB, but the root xvda uses 40 GB.

The simple solution is to run the following command on the root partition and tell it to start using the new space it has been allocated:

$ sudo growpart /dev/xvda 1

However, when I did this I was presented with the following error:

mkdir: cannot create directory ‘/tmp/growpart.2626’: No space left on device

Arrrgh! This meant that I didn’t even have enough disk space to expand the disk space. I was screwed! One of the suggestions that was mentioned online was to use the $ apt-get autoremove command to remove those dependencies that were installed with applications and are no longer used by anything else on the system. Unfortunately, I was also met with the “ No space left on device “ error when I ran the command.

I searched for other files to remove, but being a bit of an amateur with Linux, I decided to delete the safest files....the log files. I did this by typing the following command in the terminal:

$ find /var/log -type f -delete

Whew! This bought me an additional 300 Mb which was just enough to run the growpart command again.

$ sudo growpart /dev/xvda 1

Success! If I run the lsblk command to verify that partition 1 is expanded to 40 GB, I see the following:

NAME MAJ: MIN RM SIZE RO TYPE MOUNTPOINT xvda 202:0 0 40G 0 disk └─xvda1 202:1 0 40G 0 part / loop1 7:1 0 97.9M 1 loop /snap/core/10444 loop3 7:3 0 97.9M 1 loop /snap/core/10577 loop4 7:4 0 55.4M 1 loop /snap/core18/1932 loop6 7:6 0 55.4M 1 loop /snap/core18/1944

I ran the resize2fs command to resize my file system. It can be used to enlarge or shrink an unmounted file system located on device.

$ sudo resize2fs /dev/xvda1

Finally, I started the Ghost CMS again with the following command :

sudo /opt/bitnami/ctlscript.sh start

Once I ran that command I was greeted with:

ℹ Checking if logged in user is directory owner [skipped] ✓ Checking current folder permissions ✓ Validating config ✓ Checking memory availability ✓ Starting Ghost You can access your publication at http://xx.xxx.xx.x:80 Your admin interface is located at http://xx.xxx.xx.x:80/ghost/

I never thought I’d be so happy to see the output from a

You've got the Rails basics. So why do you feel so slow?

Justin Weiss

You're confident about the core ideas behind Rails. You can write working code, no problem. And you're learning more about code quality, refactoring, writing great tests, and object-oriented design.

By this point, you're starting to feel like you're getting it, that you're on the path to becoming an expert. When you look backwards, you see just how far you've come, and you're pretty happy with your progress.

So why do you feel so slow? Now that you care about testing, maintainability, and design, it feels like it takes you way more time to ship anything!

Is it even possible to ship high quality code quickly?

It's all part of the process

This feeling is incredibly common, no matter what you're learning.

Now that you're no longer a beginner, you're starting to see all the different shapes that your code could have. You have more alternatives to think through whenever you put down a line of code. You have to test edge cases you never recognized before.

You've learned lots of helpful skills. But right now, they still take a lot of thought. You have to weigh every decision you make, so you feel comfortable that you're making the right decision based on the things you've learned.

It will get faster, though. The skills you've learned will become more automatic. You'll build intuition. And you'll be able to make better decisions more quickly.

Which is nice to know, but it doesn't help you right now . So what can you do now, to finish things faster?

Take it in stages

If you're obsessed with writing perfect, high-quality, highly-maintainable code every time you put your fingers on the keyboard, you'll never get anything done.

When I get stuck, I write code the same way I write articles. You'd start with a rough draft. Maybe sketch out some tests, code, or comments. Or even write some ideas out on paper. At this point, you wouldn't worry about structure, you're just using code to clear up the vague ideas you have in your head.

Then, I turn those ideas into a straightforward implementation. What you might call "The simplest thing that could possibly work." It's not perfect, and not even close. But don't worry about it. Because once the code works, you'll do a tidying pass. TDD edge cases, refactor obviously bad code, or make names clearer.

These "refined drafts" are usually good enough to ship. But I'll usually do a few more passes. Not too many, though – you'll soon start to see diminishing returns. You'll spend more time cleaning up the code than it's worth.

Then, if you really want to end up with the cleanest possible code, let it settle for a while. Come back to it in a few weeks or months, and do another pass at it. By that time, you'll know more about your system, and you'll have learned more about how to write great, highly-maintainable code. So you'll do an even better job.

Just like writing, that process is:

Sketch out a rough outline, draft, or prototype.

Write a simple, unedited, straightforward implementation (often guided by TDD, or written along with tests).

Refine, refactor, and clean up that implementation a little bit.

Let it settle.

Come back to it, and do one more pass.

It sounds like a lot more work. But when you go in stages like this, you'll move faster, without always second-guessing yourself. And you won't end up overthinking decisions between a few just-as-good options.

This article was inspired by a question from Topher on my advice page . If you're stuck on questions about Ruby and Rails, and need some help or advice, ask me there !

Not too many, though – you'll soon start to see diminishing returns.

Justin Weiss

Chickens And Pigs

Reginald Braithwaite · Reginald Braithwaite (raganwald)

Paul Graham tweeted something interesting , and worth discussing. But first, a fabulous story, The Chicken and the Pig :

A Pig and a Chicken are walking down the road.

The Chicken says: “Hey Pig, I was thinking we should open a restaurant!”

Pig replies: “Hm, maybe, what would we call it?”

The Chicken responds: “How about ‘ham-n-eggs’?”

The Pig thinks for a moment and says: “No thanks. I’d be committed, but you’d only be involved!”

The story is usually told to illustrate the effects that differening levels of commitment have on stakeholders in a project. When writing departmental software, the manager of the department, who is accountable for its productivity, is a pig. The company’s IT manager, on the other hand, is usually a chicken.

This manifests itself in behaviour such as the IT manager demanding SharePoint compatibility, because they’re accountable for the ops personnel training budget.

Such a decision might make the software less usable for the department, but the IT Manager is not responsible for the department’s productivity, and this produces a natural tension between the two managers.

But what about the tweet?

Textbooks are not just expensive, they’re usually terrible. If they were great, the cost would be a minor concern. But when you have a medicore product, it’s natural to resent being forced to pay through the nose to own it.

terrible textbooks

The mechanism behind terrible textbooks is the same mechanism behind terrible enterprise software. In my experience, enterprise software can be effective when selected by people who don’t have to use it. The key is whether it is selected by pigs. People who have to use it are one kind of pig. People accountable for the productivity it delivers are another kind of pig. As long as an enterprise software project is driven by pigs and the chickens are restricted to an advisory capacity, you can have effective software.

But even one chicken with direct management authority or veto power can ruin an enterprise project: They “coattail” requirements that suit their own objectives at the expense of the project’s success, and you end up with nonsense. (This happens everywhere. Microsoft is suffering now because its “cash cows,” Windows and Office, behaved like chickens on steriods for decades, making almost every other division and department subordinate to their goals.)

So about textbooks. It’s fairly obvious that the problem with textbook quality and price is that the students are pigs and everyone else–faculty, administration, publishers–are chickens with authority. Fixing this problem could be done by routing around the whole system. For example, make universities irrelevant.

why we are about chickens and pigs

Regardless of what we do or do not do with textbooks, we should always pay attention to the chickens and pigs dynamic. It’s an enormously indsidious pheneomenon, and wherever you find it, you find waste and human misery.

Which also means, wherever you find it, there are opportunities to make money and to make people’s lives better. If you figure out how to disrupt textbooks, you’re cracking a billion dollar market, and helping people get an affordable education. If you figure out how to help enterprise software developers work together, better, you’re cracking another billion dollar market and again making people happy doing something they love.

So in conlusion... Textbooks are an example of the chicken and egg dynamic. It’s an important dynamic to understand, because it is very common, and if you can crack it, you can make money while making people’s lives better.

Which reminds me of another thing I once read:

“Where there’s muck, there’s brass.”—Paul Graham

Smarch

Jonathan Snook

It's mid-March and the weather has been vacillating between balmy sweater weather and the frozen tundra that need hollowed out tauntauns to stay warm. I've chosen to sequester myself indoors until some semblance of summer remains with relative consistency.

I've been using some of that time to review all things financial. With both my kids now in adulthood, I've been reviewing my estate plans. Welcome to my life.

My bank, as you might imagine, wants to sell me on trusts and estate management and insurance and all sorts of things they make money on. There is a reason why these things are worthwhile to have and under the right circumstances, I could see that I might agree to those things. Crazy.

As a dual citizen of both the US and Canada, things that work in one country don't seem to work in the other and I'm inclined to default to the easiest solution—which is, do nothing. That is to say, taxes get paid and whatever is left, if anything, goes to the kids. Easy peasy.

I still have a bunch of items to review and decide upon. The insurance was the only thing that I need to consider sooner rather than later because of the health review. As I get older, it's only going to become more expensive due to the likelihood of finding myself in worse health. I don't wanna think about getting older.

After selling off the house, living in an apartment has inspired me to simplify my life in many ways. Seeing whole swaths of items disappear off my expense spreadsheet has me wanting to pare down the expenses even more—although there's not much more I can shave down. My biggest expenses outside of taxes and living expenses are cars and accounting.

Having a car seems unnecessary at this point considering I drive maybe once or twice a month now, usually to get bulk stuff from the grocery store or to visit friends and family. I haven't quite committed to having no car at all but it's a consideration—which is funny given I just bought a new car less than two years ago. We'll see how I feel about it six months from now.

Not sure I want to do anything about the accounting. I could try to do it myself but the cross-border filings are more than I want to risk trying to figure out. And I receive enough K-1 statements to make the whole thing a hassle. I'm more likely to screw things up and I've heard too many horror stories about the IRS to want to deal with that on my own.

I also want to keep things simple for when I'm gone (hopefully way off in the future). I think about how my kids will need to go through all of my stuff and I want it to be as easy as possible for them to do that. I have documentation that I try to keep updated with all the important contacts and processes for things that will need to be dealt with.

Perhaps oddly, I worry about my domain name. Neither of my kids are technical, so maintaining domains and server management isn't their thing. I'm sure they could figure it out but I imagine that'll be one of those things that will easily fall into disrepair.

But I'll be dead. It'll be up to them to figure out if that's a problem or not.

I've also used the time to plan out future travel. I haven't had much luck with travel agents and it reminds me of every time I've used AI: It does an okay job but never a good enough job that I would think of off-loading any of that work unless for very specific purposes.

It's not necessarily about finding the cheapest price. It's about finding the right balance between comfort and location and price. That nuance is hard to explain and thus, hard for someone else to get right.

Flights and accommodations and restaurants have been booked. Schedules have been organized. I'll have that and a bit of jet lag to look forward to.

What's New, Scooby-doo.

I've been working on a new design for this very site. I'd like to get off this archaic system I've had for the past (looks at calendar) nearly twenty years. That's impressive when I think about it. Sometimes it seems like nothing changed over that time but I've looked over the original design I had when I launched and hadn't realized how much I've iterated and changed over the years. There used to be a sidebar. There used to be a hedcut. There was no green border. Much of the original design is still around, though, and I'm amazed at how well it holds up after all these years.

For pretty much every other site I run, I've switched everything over to 11ty (er, Build Awesome) and will likely turn this one over to that, too. That'll take some time.

I've got some initial designs that I like but they don't feel like they're pushing any edges and I was hoping for at least one edge to be pushed. I'll keep chipping away at it and hopefully an idea will coalesce and solidify into something I'm proud of.

I might switch everything over at a technical level before a new design is ready because I can always iterate on the design. This is my personal playground, after all.

Other than that, life is pretty simple these days and I am delightedly content with that. Given the state of the world, I feel a bit guilty expressing my personal successes, mental or monetary. I appreciate where I am right now and most days I take a moment of gratitude, slowing down time, and being present in the moment.

Reply via email

Goodbye Irida

Lazarus Lazaridis (iridakos)

On Monday I had to put my beloved cat, Irida, to sleep.

Who is Irida

Twelve years ago, I wanted to get a pet. My friends helped me find one via a post about a stray young cat which was found under a truck in my area. After two days the cat was in my house and that's how one my most beloved beings came into my life.

I was thinking of names and I ended up naming her "Irida" (which is a variation of Iris) after a server in my university. It worked well for me that everybody thought the name was given due to her special look .

This blog is named after her.

Living with Irida

If you are a cat owner you must already know that you are the only person that really knows and understands his/her behavior. Cats are not as expressive as dogs are, but sooner or later we do feel their love and we can understand their mood and their needs.

In my case, living with Irida was as if I had another person in the house. We had conversations that always ended when her look was telling me "You do know I can't speak, you're speaking alone".

As the years were passing by, our relationship was changing and even more feelings were being expressed from both parts.

Every single post of this blog has been written with this thing on the side.

Whenever I was in the house, Irida was next to me. She was following me everywhere I was going.

After the first two or three years she started "requesting" my arm to sleep on it during the night.

In the winter, she was scratching the blankets to force me elevate my hand and create some kind of a nest for her to lay down.

So many more things to say in this section. View photos instead.

Hello.

Very interesting.

Irida was happy, she was playing with her toys, purring and doing her usual crazy moves when she was bored.

Lazarus Lazaridis (iridakos)

I fits.

Bye sweater

Can't find Minerva

Who's there

I don't like reggae

Watching a thriller

The sickness

In September, while I was petting her, I noticed something like a pimple outside her mouth. After a very stressful process of tests instructed by our vet, she was diagnosed with Squamous cell Carcinoma on her jaw.

The cancer was really aggressive and we had to act fast. The doctors said that we can remove the bottom right part of the jaw which was the part that the tumor was located at. It was very hard to take a decision when you couldn't know how things would turn up. Would she be able to use her mouth afterwards? How much time would she gain? Would she be suffering after such a surgery?

We decided to have the surgery. And everything went fine.

The first days she couldn't eat on her own and we used a tube in order to feed her.

I feared that this might be it, she might not be able to use her mouth.

After 3 days she started eating on her own. And not only canned food which was soft. She started eating her dry food as if nothing had happened. The surgery went fine, the doctors did a really good job.

Everything was fine until recently. Irida was happy, she was playing with her toys, purring and doing her usual crazy moves when she was bored.

During Christmas though, she started keeping her mouth open. The scheduled tests (including scans) didn't show anything in her lungs or somewhere else. The doctors couldn't see anything developing in her mouth either. I was hoping that something else was the cause of this, like the feline asthma or the exposure to the heat of the radiators.

Few weeks ago though, she started having difficulties after eating: she was choking and she was heavily breathing. The doctors used general anesthesia and checked her larynx and that's where the problem was. There was a metastasis

Productionalizing ECS

Curalate Engineering

In January of last year we decided as a company to move towards containerization and began a migration to move onto AWS ECS . We pushed to move to containers, and off of AMI based VM deployments, in order to speed up our deployments, simplify our build tooling (since it only has to work on containers), get the benefits of being able to run our production code in a sandbox even locally on our dev machines (something you can't really do easily with AMI's), and lower our costs by getting more out of the resources we're already paying for.

However, making ECS production ready was actually quite the challenge. In this post I'll discuss:

Scaling the underlying ECS cluster

Upgrading the backing cluster images

Monitoring our containers

Cleanup of images, container artifacts

Remote debugging of our JVM processes

Which is a short summary of the things we encountered and our solutions, finally making ECS a set it and forget it system.

Scaling the cluster

The first thing we struggled with was how to scale our cluster. ECS is a container orchestrator, analogous to Kubernetes or Rancher, but you still need to have a set of EC2 machines to run as a cluster. The machines all need to have the ECS Docker agent installed on it and ECS doesn't provide a way to automatically scale and manage your cluster for you. While this has changed recently with the announcement of Fargate, Fargate's pricing makes it cost prohibitive for organizations with a lot of containers.

The general recommendation that AWS gave with ECS was to scale based on CPU reservation limit OR memory limit. There's no clear way to scale with a combination of the two, since auto scaling rules need to apply to a single CloudWatch metric or you face potential thrashing.

Our first attempt on scaling was to try and scale on container placement failures. ECS logs a message when containers are unable to be placed due to constraints (not enough memory on the cluster, or not enough CPU reservation left), but there is no way to actually capture that event programmatically (see this github issue). The goal here was to not preemptively scale, but instead scale on actual pressure. This way we wouldn't be overpaying for machines in the cluster that aren't heavily used. However we had to discard this idea since it wasn't possible due to API limitations.

Our second attempt, and one that we have been using now in production, is to use an AWS Lambda function to monitor the memory and CPU reservation of the cluster and emit a compound metric to CloudWatch that we can scale on. We set a compound threshold with the logic of:

If either memory or CPU is above the max threshold, scale up

Else if both memory and CPU are below the min, scale down.

Else stay the same

We represent a scale up event with a CloudWatch value of 2 , a scale down as value 0 and otherwise the nominal state as value 1 .

The code for that is shown below:

package com.curalate.lambdas.ecs_scaling

import com.amazonaws.services. CloudWatch. AmazonCloudWatch
import com.amazonaws.services. CloudWatch.model._ import com.curalate.lambdas.ecs_scaling. ScaleMetric. ScaleMetric import com.curalate.lambdas.ecs_scaling.config. ClusterScaling import org.joda.time. DateTime import scala.collection. JavaConverters._

object ScaleMetric extends Enumeration { type ScaleMetric = Value

val ScaleDown = Value (0) val StaySame = Value (1) val ScaleUp = Value (2) }

case class ClusterMetric (clusterName : String , scaleMetric : ScaleMetric , periodSeconds : Int)

class MetricResolver (clusterName : String , cloudWatch : AmazonCloudWatch) { private lazy val now = DateTime . now () private lazy val start = now . minusMinutes (3)

private val dimension = new Dimension (). withName ("ClusterName"). withValue (clusterName)

val periodSeconds = 60

protected def getMetric (name : String) : Double = { val baseRequest = new GetMetricStatisticsRequest (). withDimensions (dimension)

cloudWatch . getMetricStatistics (baseRequest . withMetricName (name). withNamespace ("AWS/ECS"). withStartTime (start . toDate). withEndTime (now . toDate). withPeriod (periodSeconds). withStatistics (Statistic . Maximum)). getDatapoints . asScala . head . getMaximum }

lazy val currentCpuReservation : Double = getMetric ("CPUReservation")

lazy val currentMemoryReservation : Double = getMetric

Azure Function Timer Trigger not firing - NCrontab

Dean Hume

If you've recently ventured into the world of Azure Functions , you're likely familiar with the versatility they offer when it comes to scheduling tasks. In this guide, we'll delve into using Azure Functions' Time Trigger function with Node.js and Visual Studio Code, uncovering some important nuances along the way.

This article assumes that you have some familiarity with Node.js and a basic understanding of Azure.

To begin, you'll need Node.js and Visual Studio Code installed, along with the Azure Functions extension. When you create a new function, it comes with some boilerplate code that includes a schedule property defining when the function should run. By default, it's set to run every 5 minutes.

const { app } = require('@azure/functions');

The schedule parameter in the above code tells the Azure Function how often to fire.

But what if you want your function to run every 11 hours during a day? Crafting the right Cron expression can be tricky, and that's where I encountered some difficulties. I turned to an online Cron expression generator, hoping it would simplify the process. Here's what it gave me:

0 0 */11 * *

Feeling confident, I integrated this expression into my code above and deployed the Azure Function. However, it didn't work as expected. I scoured the logs for clues but found nothing. To troubleshoot and debug locally, I temporarily changed the Cron expression to run every minute, and it worked flawlessly. So why wasn't it firing every few hours?

After some online research, I uncovered the reason behind the issue. Azure Functions use an NCRONTAB expression , which requires a six-part format instead of the traditional five-part Cron expression. The online generator that I found online had provided a five-part expression, leading to the problem.

To be fair, the Visual Studio Code extension for Azure Functions provides the correct six-part format by default. My mistake was changing it using a five-part expression from an online tool - d'oh!

To rectify this, I recommend using the NCrontab Expression Tester . This user-friendly tool not only helps you test your expressions but also generates the correct six-part Cron expressions tailored for Azure Functions.

When I updated my code to use the 6 part format:

0 0 */11 * * *

It ran perfectly!

Why NCrontab?

You might be wondering why Azure Functions use the NCrontab six-part format instead of the traditional five-part format. The answer lies in its flexibility. The six-part format allows you to specify seconds, enabling you to run your functions with higher precision and frequency.

* * * * * *

• ▸ – • ▸ –

| | | | | | | | | | | +— day of week (0 - 6) (Sunday=0) | | | | +—– month (1 - 12) | | | +——- day of month (1 - 31) | | +——— hour (0 - 23) | +———– min (0 - 59) +———–- sec (0 - 59)

For further details on NCrontab expressions, you can visit the GitHub repository here .

Happy coding!

Exploring Structural Sharing and Copy-on-Write Semantics, Part I

Reginald Braithwaite · Reginald Braithwaite (raganwald)

This essay takes a highly informal look at two related techniques for achieving high performance when using large data structures: Structural Sharing , and Copy-on-Write Semantics . In Part I, we'll look at the background of Structural Sharing and start making a Slice class that abstracts the concept of a slice of an array. In Part II , we'll consider the problem of resource ownership when mutating objects.

To give us some context for exploring these techniques, we're going to solve a very simple problem: Programming in a Lisp-like recursive style, but using JavaScript arrays. Although not the most practical use case, it's interesting because even a small function written in Lisp style (like summing a list of integers) can create and recycle a lot of temporary objects.

Fixing that problem gives us an excuse to look at ways to use memory efficiently, minimizing the data we have to copy. Although we're unlikely to use recursion gratuitously to sum a list of integers, the techniques we'll use here to make it "not embarrassing" are the exact same techniques needed to make working with large data structures performant.

And now, let's start at the beginning. The beginning of functional programming, in fact.

wherein we travel back in time to the dawn of functional programming

Once upon a time, there was a programming language called Lisp , an acronym for LIS(t) P(rocessing). 1 Lisp was one of the very first high-level languages, the very first implementation was written for the IBM 704 computer. 2

The 704 had a 36-bit word, meaning that it was very fast to store and retrieve 36-bit values. The CPU's instruction set featured two important macros: CAR would fetch 15 bits representing the Contents of the Address part of the Register, while CDR would fetch the Contents of the Decrement part of the Register.

In broad terms, this means that a single 36-bit word could store two separate 15-bit values and it was very fast to save and retrieve pairs of values. If you had two 15-bit values and wished to write them to the register, the CONS macro would take the values and write them to a 36-bit word.

Thus, CONS put two values together, CAR extracted one, and CDR extracted the other. Lisp's basic data type is often said to be the list, but in actuality it was the "cons cell," the term used to describe two 15-bit values stored in one word. The 15-bit values were used as pointers that could refer to a location in memory, so in effect, a cons cell was a little data structure with two pointers to other cons cells.

Lists were represented as linked lists of cons cells, with each cell's head pointing to an element and the tail pointing to another cons cell.

Having these instructions be very fast was important to those early designers: They were working on one of the first high-level languages (COBOL and FORTRAN being the others), and computers in the late 1950s were extremely small and slow by today's standards. Although the 704 used core memory, it still used vacuum tubes for its logic. Thus, the design of programming languages and algorithms was driven by what could be accomplished with limited memory and performance.

Here's the scheme in JavaScript, using two-element arrays to represent cons cells:

```
const cons = ( a , d ) => [ a , d ], car = ([ a , d ]) => a , cdr = ([ a , d ]) => d ;
```

We can make a list by calling cons repeatedly, and terminating it with null :

```
const oneToFive = cons ( 1 , cons ( 2 , cons ( 3 , cons ( 4 , cons ( 5 , null )))));
```

Notice that though JavaScript displays our list as if it is composed of arrays nested within each other like Russian Dolls, in reality the arrays refer to each other with references, so [1,[2,[3,[4,[5,null]]]]] is our way to represent:

This is a Linked List , it's just that those early Lispers used the names car and cdr after the hardware instructions, whereas today we use words like element and next . But it works the same way: If we want the head of a list, we call car on it:

car is very fast, it simply extracts the first element of the cons cell. And what about the rest of the list? cdr does the trick:

That's another linked list too:

By extracting references from cons cells, it achieves high performance. In Lisp, it's blazingly fast because it happens in hardware. There's no making copies of arrays, the time to get the cdr of a list with five elements is the same as the time to get the cdr pf a list with 5,000 elements. In each case, we have a reference to the first cell, and we get a reference to the next cell in one step. No elements need to be copied and the list does not need to be traversed.

In JavaScript, even without the low-level support, it's still much, much, much faster to get all the elements except the head from a linked list than from an array. Getting one reference to a structure that already exists is faster than copying a bunch of elements.

So now we understand that in Lisp, a lot of things use linked lists, and they do that in part because it was what the hardware made fast.

Symbolics, Inc. was a computer manufacturer headquartered in Cambridge, Massachusetts, and later in Concord, Massachusetts, with manufacturing facilities in Chatsworth, California. Symbolics designed and manufac-

Mountains

Lazarus Lazaridis (iridakos)

After painting the meadow with acrylic colors , I bought oil colors which are the ones Bob Ross uses in “The joy of painting” show. This time I followed the episode 10 from season 13 of the show, called “Mountain Summit” .

I am very satisfied by the outcome but being a beginner, I faced a lot of difficulties and the painting has a number of errors.

I used

the wet-on-wet technique

oil colors

50x80 canvas

My paintbrushes were not the same as Bob Ross’ ones (size and material) and the paint didn’t apply the way I wanted (I will search for tips and tricks when it comes to buying paintbrushes for oil colors)

I couldn’t find all the colors listed in the episode and I had to mix mine many times to achieve something similar

I had such a huge difficulty creating the bushes. I am still not sure what the exact problem was but I believe it is a combination of the following three items:

wrong paintbrush

very thick paint (according to Bob Ross, thin paint sticks on thick)

wrong moves by me (maybe more/less pressure on the paintbrush)

The mountains, even though they look nice, were so hard to be created with the palette knife. It surely has to do with practice.

The tallest mountain has an unnatural curve on its bottom side which doesn’t make sense

The left side of the mountains ends somewhere behind the trees without a logical explanation (in the episode the trees cover all the left part of the painting)

The path in the front right section of the painting is a failure. In case you don’t locate it (which is very reasonable), I am talking about the black and gray waterfall that starts at the root of a tree. Yes, that is supposed to be a path.

In the same part of the painting, the bushes didn’t want to stick at all. I cleaned and re-painted the section at least 3 times.

Bob Ross - Mountain Summit (Season 13 Episode 10)

https://www.youtube.com/watch?v=kasGRkfkiPM

Bob Ross - Final Reflections (Season 1 Episode 13)

A “must see” episode because it’s actually Q&A of what might go wrong.

https://www.youtube.com/watch?v=IEQWfszfRlA

ANALYSIS

CSS <code>@scope</code>: An Alternative To Naming Conventions And Heavy Abstractions

Blake Lundquist · Smashing Magazine

When learning the principles of basic CSS, one is taught to write modular, reusable, and descriptive styles to ensure maintainability. But when developers become involved with real-world applications, it often feels impossible to add UI features without styles leaking into unintended areas.

This issue often snowballs into a self-fulfilling loop; styles that are theoretically scoped to one element or class start showing up where they don't belong. This forces the developer to create even more specific selectors to override the leaked styles, which then accidentally override global styles, and so on.

Rigid class name conventions, such as BEM , are one theoretical solution to this issue. The BEM (Block, Element, Modifier) methodology is a systematic way of naming CSS classes to ensure reusability and structure within CSS files. Naming conventions like this can reduce cognitive load by leveraging domain language to describe elements and their state , and if implemented correctly, can make styles for large applications easier to maintain .

In the real world, however, it doesn't always work out like that. Priorities can change, and with change, implementation becomes inconsistent. Small changes to the HTML structure can require many CSS class name revisions. With highly interactive front-end applications, class names following the BEM pattern can become long and unwieldy (e.g., app-user-overview__status–is-authenticating), and not fully adhering to the naming rules breaks the system's structure, thereby negating its benefits.

Given these challenges, it's no wonder that developers have turned to frameworks, Tailwind being the most popular CSS framework . Rather than trying to fight what seems like an unwinnable specificity war between styles, it is easier to give up on the CSS Cascade and use tools that guarantee complete isolation.

Developers Lean More On Utilities

How do we know that some developers are keen on avoiding cascaded styles? It's the rise of "modern" front-end tooling — like CSS-in-JS frameworks — designed specifically for that purpose. Working with isolated styles that are tightly scoped to specific components can seem like a breath of fresh air. It removes the need to name things — still one of the most hated and time-consuming front-end tasks — and allows developers to be productive without fully understanding or leveraging the benefits of CSS inheritance.

But ditching the CSS Cascade comes with its own problems. For instance, composing styles in JavaScript requires heavy build configurations and often leads to styles awkwardly intermingling with component markup or HTML. Instead of carefully considered naming conventions, we allow build tools to autogenerate selectors and identifiers for us (e.g., .jsx-3130221066), requiring developers to keep up with yet another pseudo-language in and of itself. (As if the cognitive load of understanding what all your component's useEffect s do weren't already enough!)

Further abstracting the job of naming classes to tooling means that basic debugging is often constrained to specific application versions compiled for development, rather than leveraging native browser features that support live debugging, such as Developer Tools.

It's almost like we need to develop tools to debug the tools we're using to abstract what the web already provides — all for the sake of running away from the "pain" of writing standard CSS.

Luckily, modern CSS features not only make writing standard CSS more flexible but also give developers like us a great deal more power to manage the cascade and make it work for us. CSS Cascade Layers are a great example, but there's another feature that gets a surprising lack of attention — although that is changing now that it has recently become Baseline compatible .

The CSS @scope At-Rule

I consider the CSS @scope at-rule to be a potential cure for the sort of style-leak-induced anxiety we've covered, one that does not force us to compromise native web advantages for abstractions and extra build tooling.

"The @scope CSS at-rule enables you to select elements in specific DOM subtrees, targeting elements precisely without writing overly-specific selectors that are hard to override, and without coupling your selectors too tightly to the DOM structure."

— MDN

In other words, we can work with isolated styles in specific instances without sacrificing inheritance, cascading, or even the basic separation of concerns that has been a long-running guiding principle of front-end development.

Plus, it has excellent browser coverage . In fact, Firefox 146 added support for @scope in December, making it Baseline compatible for the first time. Here is a simple comparison between a button using the BEM pattern versus the @scope rule:

Click me →

.button .button__text { /* button text styles */ } .button .button__icon { /* button icon styles */ } .button–primary { primary button styles */ }

Click me →

@scope (.primary-button) { span:first-child { /* button text

A JavaScript Constructor Problem, and Three Solutions

Reginald Braithwaite · Reginald Braithwaite (raganwald)

preamble

As you know, you can create new objects in JavaScript using a Constructor Function , like this:

function Fubar (foo , bar) { this . _foo = foo ; this . _bar = bar ; }

var snafu = new Fubar (" Situation Normal " , " All Fsked Up ");

When you "call" the constructor with the new keyword, you get a new object allocated, and the constructor is called with the new object as the current context. If you don't explicitly return anything from the constructor, you get the new object as the result.

Thus, the body of the constructor function is used to initialize the newly created object. There's another thing: The newly created object is initialized to have a prototype. What prototype? The contents of the constructor's prototype property. So we can write:

Fubar . prototype . concatenated = function () { return this . _foo + " " + this . _bar ; }

Thanks to the internal mechanics of JavaScript's instanceof operator, we can use it to test whether an object is likely to have been created with a particular constructor:

(It's possible to "fool" instanceof when working with more advanced idioms, or if you're the kind of malicious troglodyte who collects language corner cases and enjoys inflicting them on candidates in job interviews. But it works well enough for our purposes.)

the problem

What happens if we call the constructor, but accidentally omit the new keyword?

var fubar = Fubar (" Fsked Up " , " Beyond All Recognition ");

William-Thomas-Fredreich!? We've called an ordinary function that doesn't return anything. so fubar is undefined. That's not what we want. Actually, it's worse than that:

JavaScript sets this to the global environment by default for calling an ordinary function, so we've just blundered about in the global environment. We can fix that somewhat:

function Fubar (foo , bar) { " use strict "

this . _foo = foo ; this . _bar = bar ; }

Although "use strict" might be omitted from code in blog posts and books (mea culpa!), in production it is very nearly mandatory for reasons just like this. But nevertheless, constructors that do not take into account the possibility of being called without the new keyword are a potential problem.

So what can we do?

solution: auto-instantiation

In Effective JavaScript , David Herman describes auto-instantiation . When we call a constructor with new , The pseudo-variable this is set to a new instance of our "class," so-to-speak. We can use this to detect whether our constructor has been called with new :

function Fubar (foo , bar) { " use strict "

var obj , ret ;

if (this instanceof Fubar) { this . _foo = foo ; this . _bar = bar ; } else return new Fubar (foo , bar); }

Why bother making it work without new ? One problem this solves is that new Fubar(...) does not compose . Consider:

function logsArguments (fn) { return function () { console . log . apply (this , arguments); return fn . apply (this , arguments) } }

function sum2 (a , b) { return a + b ; }

var logsSum = logsArguments (sum2);

logsArguments decorates a function by returning a version of the function that logs its arguments. Let's try it on the original Fubar :

function Fubar (foo , bar) { this . _foo = foo ; this . _bar = bar ; } Fubar . prototype . concatenated = function () { return this . _foo + " " + this . _bar ; }

var LoggingFubar = logsArguments (Fubar);

This doesn't work because snafu is actually an instance of LoggingFubar , not of Fubar . But if we use the auto-instantiating version of Fubar :

function Fubar (foo , bar) { " use strict "

var obj , ret ;

if (this instanceof Fubar) { this . _foo = foo ; this . _bar = bar ; } else { obj = new Fubar (); ret = Fubar . apply (obj , arguments); return ret === undefined ? obj : ret ; } } Fubar . prototype . concatenated = function () { return this . _foo + " " + this . _bar ; }

var LoggingFubar = logsArguments (Fubar);

Now it works, but of course snafu is an instance of Fubar , not of LoggingFubar . Is that what you want? Who knows!? This isn't a justification for the pattern, as much as an explanation that it is a useful, but leaky abstraction. It's

How to write better emails

Lazarus Lazaridis (iridakos)

Email communication is not my favorite but since I can't avoid it, I am trying to compose messages in a way that I think it makes it easier for both me and the recipient:

to quickly address what is being communicated

avoid misunderstandings

save time

Here are some tips. They don't apply to all type of messages, I provide before and after examples to better describe each case.

Emphasize text with bold/underlined font

Emphasizing the appropriate parts of a message, especially when it's a long one, you help readers quickly get an idea of what the email is about and easily locate the important stuff after going back to it at some point in the future.

Hello all,

I noticed that there are many logs for blabla the last few days and I don't think that it is normal. I believe the problem is the updated version of gem blabla.

I have opened an issue describing the case in Redmine (#455) in the current version. Feel free to change its priority in case blabla.

Thanks, Lazarus

Hello all,

I noticed that there are many logs for blabla the last few days and I don't think that it is normal. I believe the problem is the updated version of gem blabla.

I have opened a Redmine issue (#455) describing the case in the current version. Feel free to change its priority in case blabla.

Thanks, Lazarus

Use specific dates instead of yesterday , tomorrow etc

The moment you send an email is not the moment that it will be read by its recipients. Avoid using only temporal adverbs/nouns like yesterday , today , tomorrow , two hours ago etc but include also the specific dates/times otherwise they might be misunderstood or require from recipients to check the email's sent date/time to calculate the actual time.

Dear QA,

Yesterday we released a fix for the bug 455 on staging and we plan to release it next Monday if you give us the green light by tomorrow end of day.

Thanks, Lazarus

Dear QA,

Yesterday , June 25th, 2019 we released a fix for the bug #455 on staging and we plan to release it on production next Monday (July 1st, 2019) if you give us the green light by tomorrow (June 27th, 2019) end of day.

Thanks, Lazarus

Use links for references

Use bookmarkable links when you refer to something that would eventually require from the recipient to search for in another platform.

This has two benefits:

Save time

Eliminate ambiguous references

Dear Phoebe,

I didn't understand the process described on the issue about the logging bug. Irida's comment was not very clear either. Can you please help me?

Thanks, Lazarus

Dear Phoebe,

I didn't understand the process described on the issue about the logging bug (Redmine #453). This comment from Irida was not very clear either. Can you please help me?

Thanks, Lazarus

Structure long messages

Long messages are in general not very effective and parts of them are prone to be ignored.

When I have to compose such a message I try to categorize the text in contextual sections and then structure them using headers and paragraphs allowing readers to navigate to them at a glance.

Dear team,

Last week we had a problem with the logs in the production environment. I am talking about the Redmine issue #453. We noticed a huge increase in the log messages blabla leading to delayed responses because blah blah blah. At some point the server run out of disk and everything fell apart blah blah blah. Administrators backed up the logs and removed them from the server to blah blah blah. We started investigating what is going on immediately and after two days we managed to reproduce the error in the staging environment as well. The problem was the usage of an external library that had a bug which blah blah blah. We removed

A chess day

Petr Mitrichev · Petr Mitrichev (competitive programming)

Yesterday was the day of the Dress Rehearsal and the Tech Trek at #ICPCBaku. I am planning to solve the mirror contest on Thursday together with Kevin and Mateusz , so we used the Dress Rehearsal time to practice our solving and streaming setup as well (photo on the left). Unfortunately we could not practice the submissions since it seems that the mirror was not available for the Dress Rehearsal, but the rest seems in order. Tune in tomorrow (Thursday) at 11:00 Baku time to my Twitch to watch us compete! You will also be able to look at the official broadcast of the real contest, its scoreboard and problems (you can find more links here), and of course please support the #ETH team!

Today is the ICPC Challenge day, with the results due to be announced in a couple of hours. In the evening the Alumni-hosted events will continue, and I have decided to use my slot for an online chess tournament for the World Finals onsite attendees. Here is how I propose to run it:

We will run a 5-round Swiss tournament on lichess with 3+0 time control (3 minutes for each side per game, no increment). This yields a 30-minute duration but if games finish earlier hopefully it will be a bit faster and everybody will be in time for the push-ups :)

To join the tournament, you need a lichess account and a phone or a laptop to play. If you do not have a lichess account and want to participate, please create it in advance!

To join the tournament, you also need to join this team on lichess first. I will need to approve everybody who wants to join the team so that it keeps being an onsite event, so you will need to find me in person for that.

The easiest way to find me in person will be to come to the Chill Zone in the Marriott Blvd today at 17:30 (or a bit earlier).

The tournament will start at 17:35, so don't be late! If you are late, you should be able to join the tournament late during the first two rounds as well, but then you will miss some games :)

If you have any questions, please ask in comments here or on Codeforces!

The danger of glamourizing one shots

Scott Hanselman

People should not be judging AI-augmented coding by "1 shots."

If someone told you that their model did a "one shot of Minecraft" and they're impressed by that, you need to consider how much semantic heavy lifting the word "Minecraft" is doing in that prompt.

Ask them to one shot Minecraft without using the word Minecraft.

It's not trivial to one shot something unique, because programming is the art of making the ambiguous incredibly specific through sculpting. AI sculpting is less about vibes and more about finding the specificity you want and keeping the system stable through changes. Good SDLC practices still matter, historical context still matters, and knowing how things work matters, shout out to Grady Booch.

It's a cool party trick to one shot Mario Brothers or space invaders, but then you'll end up with the most mid version of both. Literally mid. You'll get the statistical fat part of the Bell curve version of these mythical games. You're telling the model to close its eyes and draw the face of these games from memory.

As high-level programming cedes way to the prose compiler, making your goals and specs well understood to the ambiguity loop and showing good judgment is going to matter more than ever. Consider all of your words and make sure that certain words aren't carrying all the semantic load, hidden or otherwise.

BRIEFS

IN BRIEF

DeepMind Blog, DeepMind Blog: Gemini 2.5 Flash is our first fully hybrid reasoning model, giving developers the ability to turn thinking on or off.

DeepMind Blog, DeepMind Blog: We partnered with Darren Aronofsky, Eliza McNitt and a team of more than 200 people to make a film using Veo and live-action filmmaking.

Trivago Tech, Trivago Tech: At trivago we use Jenkins as our main CI tool. However, when our physical setup was not enough we needed to move it to the cloud and implement an automated slave scaling. This is the definite guide with all the steps we took to implement an auto scaling Jenkins platform.

Trivago Tech, Trivago Tech: When I joined trivago a year ago we had problems with our releases. Read how we were able to switch from our bash release process to a new one.

Real Python, Real Python: When it comes to Python package managers, the choice often comes down to uv vs pip . You may choose pip for out-of-the-box availability, broad compatibility, and reliable ecosystem support. In contrast, uv is worth considering if you prioritize fast installs, reproducible environments, and clean uninstall behavior, or if you want to streamline workflows for new projects.

In this video course, you'll compare both tools. To keep this comparison meaningful, you'll focus on the overlapping features, primarily package installation and dependency management .

DeepMind Blog, DeepMind Blog: Gemini 2.0 Flash-Lite is now generally available in the Gemini API for production use in Google AI Studio and for enterprise customers on Vertex AI

Trivago Tech, Trivago Tech: It has been about a year since we introduce the idea of guilds in trivago Software Engineering department in Düsseldorf. Here we share some of our learnings with it.

DeepMind Blog, DeepMind Blog: Genie 3 can generate dynamic worlds that you can navigate in real time at 24 frames per second, retaining consistency for a few minutes at a resolution of 720p.

Trivago Tech, Trivago Tech: Sometimes advertisements just have to be bold. Especially when your goal is to hire creative thinkers, why not give them something to... think about!? That was our goal when we planned a new marketing campaign for Software Engineers.

Supabase Blog, Supabase Blog: Build a professional deployment workflow for your Supabase project. Learn essential patterns that prevent 3am panic attacks while keeping your workflow fun, simple, and safe.

Supabase Blog, Supabase Blog: Today, PostgREST 10 was released. Let's take a look at some of the new features that go hand in hand with supabase-js v2.

Supabase Blog, Supabase Blog: Learn how the team at Epsilon3 use Supabase to help teams execute secure and reliable operations in an industry that project spend runs into the billions.

The Star

Vol. 1, No. 004 · 2026-03-30

Curated and composed automatically.